

Kleene Closure Property Path Query Optimization Based on Node Clustered Index

Tenglong Ren

School of Computer Software, Tianjin University
Tianjin, China
tenglongren@tju.edu.cn

Xiaowang Zhang*

School of Computer Software, Tianjin University
Tianjin, China
xiaowangzhang@tju.edu.cn

Lulu Yang

School of Computer Software, Tianjin University
Tianjin, China
luluyang@tju.edu.cn

Zhiyong Feng

School of Computer Software, Tianjin University
Tianjin, China
zyfeng@tju.edu.cn

Abstract

The evaluation of Kleene Closure Property Path Queries (KPPQs) over RDF graphs presents a fundamental trade-off. Under the intuitive *simple path semantics*, which forbids node repetition, the problem is NP-complete. This forces practical systems to adopt *existential path semantics*, which admits tractable evaluation but produces result sets bloated with redundant and often semantically meaningless paths due to unnecessary detours through **cycles**.

This paper introduces a novel indexing framework that effectively navigates this trade-off. Our key insight is that while enumerating *all* simple paths is intractable, a carefully selected, compact set of *representative simple paths* is sufficient to correctly and completely answer connectivity queries *in practice*. We propose the **Node Clustered Index (NCI)**, which pre-materializes such paths—including the longest acyclic chains and fundamental **cycles**—transforming online query processing from a costly graph traversal into an efficient index lookup and expansion operation. Crucially, our method **guarantees** that every result conforms to simple path semantics. Extensive experiments demonstrate that our approach achieves near-perfect recall, outperforming state-of-the-art systems by up to an order of magnitude. This work establishes that for NP-complete path problems, seeking polynomial-time, empirically perfect approximations is a viable and powerful paradigm, delivering both performance and quality without compromise.

CCS Concepts

• **Information systems** → Hierarchical data models; Network data models.

Keywords

SPARQL, Property Path, Kleene Closure, Node Clustered Index

ACM Reference Format:

Tenglong Ren, Lulu Yang, Xiaowang Zhang, and Zhiyong Feng. 2026. Kleene Closure Property Path Query Optimization Based on Node Clustered Index.

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792534>

In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages.
<https://doi.org/10.1145/3774904.3792534>

1 Introduction

The introduction of property paths in SPARQL 1.1 [14] marked a significant advancement in RDF query capabilities, enabling regular expression-like pattern matching over graph structures. Among these operators, the Kleene closure (*) stands out for its ability to express recursive relationships, making it indispensable for applications ranging from social network analysis to biological pathway discovery and knowledge graph completion [1, 17, 28]. Not surprisingly, Kleene closure property path queries (KPPQs) represent a substantial portion of real-world SPARQL workloads [8, 35].

However, this expressive power comes at a well-known computational cost [2, 18, 20, 23, 30]. The complexity of evaluating KPPQs depends entirely on the underlying path semantics. Under *simple path semantics*—which intuitively prohibits node repetition—KPPQ evaluation is NP-complete [20, 23]. This complexity manifests practically: query engines must maintain extensive state to avoid infinite loops in cyclic graphs, while naive traversal algorithms waste resources exploring exponentially many path permutations.

Consider the enterprise social network in Fig. 1. Evaluating the query for *all people reachable from Chen through colleague* under simple path semantics becomes computationally challenging due to cycles like $\{Chen \rightarrow Lee, Lee \rightarrow Philip, Philip \rightarrow Chen\}$. The system must navigate an exponential number of simple paths while avoiding redundant traversals.

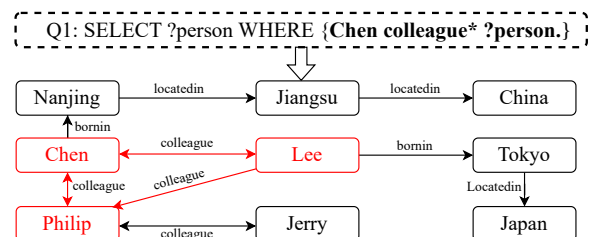


Figure 1: Example of KPPQ

Faced with this fundamental challenge, practical SPARQL engines have universally adopted *existential path semantics* [3], which

reduces the problem to polynomial time by permitting arbitrary node repetitions. However, this computational convenience exacts a heavy toll on result quality. Existential semantics produces bloated result sets filled with counterintuitive and semantically problematic paths.

Crucially, it can even create connections that do not exist in the original graph's simple-path connectivity. For instance, in our example graph, there is no simple path from *Chen* to *Jerry* via the *colleague* relationship, as they are disconnected in the simple-path view. However, under existential semantics, paths like *Chen* → *Lee* → *Philip* → *Chen* → *Philip* → *Jerry* are considered valid. This path, which repeats nodes *Chen* and *Philip*, fabricates a spurious connection between *Chen* and *Jerry* that has no basis in the genuine organizational structure. These artificially generated paths pollute the result set, obscure the true (dis)connectivity, and severely complicate any downstream processing that relies on accurate relationship information.

Current optimization approaches fail to resolve this tension. Methods that precompute paths up to fixed length k [12, 13] are fundamentally limited—they cannot handle longer paths and provide incomplete representations of cycles. Automata-based filtering [34] shifts complexity to query time, incurring substantial overhead. Recent work on compact path representation [21] focuses on output compression rather than optimizing connectivity checks.

Our approach is built on a foundational insight: for determining connectivity, a polynomial-sized subset of *representative simple paths* can effectively substitute for the exponential complete set. This transforms the problem from finding *all paths* to finding the *right paths*.

We introduce the **NCI**, a novel indexing structure that materializes these representative paths. The NCI is constructed using practical heuristics that identify longest acyclic chains and capture cyclic connectivity through pattern-aware cycle detection. This framework delivers a decisive advantage: (1) **Semantic Fidelity**: By operating exclusively over simple paths, NCI guarantees that all returned connections are semantically valid and interpretable, entirely avoiding the spurious results of existential semantics. (2) **Polynomial-Time Execution**: Query processing is reduced to efficient index lookups and expansions. We provide the first theoretical proof that online query evaluation, powered by the NCI, achieves polynomial time complexity. (3) **Practical Completeness**: While our path selection is heuristic-driven, it achieves *empirical completeness*—capturing essential connectivity patterns with recall rates that meet or exceed practical needs on real-world graphs. Our specific contributions are as follows:

- We propose the concept of **representative simple paths** and instantiate it in the **NCI**, complete with efficient construction algorithms based on graph structural patterns.
- We provide a comprehensive complexity analysis, proving polynomial bounds for both index construction and, crucially, for online query execution.
- We design a **universal decomposition model** and an **expansion module** that together enable efficient evaluation of complex KPPQs using the NCI.

- We implement our approach in a prototype system, **NKPP**. Extensive experiments show that NKPP outperforms state-of-the-art systems by up to an order of magnitude, while delivering superior, semantically precise results.

This work champions a **practically perfect** paradigm for KPPQ evaluation. We demonstrate that through careful engineering and heuristic design, it is possible to bypass theoretical intractability without sacrificing semantic quality. The NCI framework offers a viable third path, moving beyond the traditional compromise between computational hardness and meaningless results.

The paper is organized as follows. Section 2 summarizes the related work. Section 3 describes the basic definitions relevant to the work in this paper. Section 4 describes the Node Clustered Index. Section 5 describes the Expansion Module and Decomposition Model. Section 6 shows the experimental results, and we conclude in Section 7.

2 Related Work

Kleene closures enrich query language expressiveness and support SPARQL semantic extensions [2, 10, 11, 22, 27, 36]. Our work addresses the performance-semantic trade-off in evaluating Kleene closures at the intersection of path query processing and RDF systems.

2.1 Theoretical Foundations and Semantic Trade-offs

Regular path queries under *simple path semantics* are NP-complete [20, 23]. This led SPARQL to adopt *existential semantics* [3], trading semantic precision for polynomial-time evaluation. Our work bridges this divide by delivering simple-path-conformant results with practical efficiency.

2.2 Specialized Indexing Techniques

Length-bounded indexing dominates path query optimization. The k -index [12] and systems like FERRARI [13] precompute reachability for paths up to length k . Variants including FPIRPQ [31] and extended property paths [10] share this limitation: they cannot naturally represent unlimited connectivity. Our NCI handles arbitrary-length paths and cyclic structures through representative path selection.

2.3 Online Processing and Automation

Automata-based methods [34] and graph reduction techniques [24] address path queries through online processing. While expressive, these incur substantial runtime overhead from filtering large intermediate results [4]. Our approach shifts computational burden to offline indexing, enabling polynomial-time online execution.

2.4 Compact Path Representation

Martens et al. [21] developed methods for linearly encoding complete path sets. However, their focus is *output-oriented*—optimizing path enumeration. NCI is *execution-oriented*: selecting minimal path sets sufficient for connectivity queries, prioritizing execution efficiency over complete representation.

2.5 Industrial Practice and Pragmatic Solutions

RDF systems like Jena [9], Blazegraph [6], and Virtuoso [7] implement KPPQ evaluation through existential semantics with basic cycle detection, prioritizing scalability over semantic precision. This gap between academic advances and industrial practice motivates our experimental comparison.

2.6 Positioning Our Contribution

Existing approaches force a compromise: bound path length [13, 25, 26, 30, 31, 33] and accept incomplete results, or employ expensive online processing [34]. NCI offers a novel synthesis through *representative simple paths*, handling arbitrary paths naturally while shifting complexity to offline indexing. Unlike industrial systems, we deliver semantically precise results without sacrificing performance, advancing *empirically perfect approximations* with guarantees on both result quality and practical efficiency.

3 Preliminaries

3.1 RDF Graph

Definition 3.1 (RDF Graph). An **RDF graph** $G = (V, E, \Sigma)$ is defined as:

- $V = s(G) \cup o(G)$ is the set of nodes (subjects and objects)
- $\Sigma = p(G)$ is the set of edge labels (predicates)
- $E = \{(s, o, p) \mid (s, p, o) \in G\}$ is the set of labeled edges

where each triple $(s, p, o) \in (U \cup B) \times U \times (U \cup B \cup L)$ represents a directed edge $s \xrightarrow{p} o$, with U , B , and L denoting URIs, blank nodes, and literals respectively.

$s(G)$, $p(G)$, $o(G)$ denote the subject, predicate, and object sets respectively. We define an infinite set of variables Var , disjoint from U and L .

As shown in Fig. 2, the RDF graph G_0 consists of 12 triples, where $V_{G_0} = \{v_i \mid 1 \leq i \leq 9\}$, $\Sigma_{G_0} = \{e_1, e_2, e_3\}$.

3.2 Path

Definition 3.2 (Path). A **path** ρ from node u to node v in a graph G is a sequence:

$$\rho = v_0 \xrightarrow{l_0} v_1 \xrightarrow{l_1} \dots \xrightarrow{l_{n-1}} v_n$$

where $v_0 = u$, $v_n = v$, and each $(v_k, l_k, v_{k+1}) \in E$ for $k = 0, \dots, n-1$. The **label sequence** of ρ is $\lambda(\rho) = l_0 l_1 \dots l_{n-1} \in \Sigma^*$, and its **length** is $|\rho| = n$.

A path is **simple** if all nodes $v_0, \dots, v_n$ are distinct. It is a **cycle** if $u = v$ and $|\rho| > 0$.

Example 3.3. In G_0 , the path $\rho = v_7 \xrightarrow{e_1} v_6 \xrightarrow{e_1} v_1 \xrightarrow{e_1} v_5 \xrightarrow{e_1} v_8$ has label sequence $\lambda(\rho) = e_1 e_1 e_1 e_1$ and length $|\rho| = 4$. It traverses the edges (v_7, e_1, v_6) , (v_6, e_1, v_1) , (v_1, e_1, v_5) , and (v_5, e_1, v_8) .

3.3 Property Path Expressions

Definition 3.4 (Property Path Expression). A **property path expression** e is defined recursively:

$e ::= a \mid e^- \mid e_1 \cdot e_2 \mid e_1 \mid e_2 \mid e^+ \mid e^* \mid e? \mid \{a_1, \dots, a_k\} \mid \{a_1^-, \dots, a_k^-\}$
where $a, a_1, \dots, a_k \in U$ are URIs.

This paper focuses on the evaluation of expressions containing Kleene closure operators (e^*).

3.4 Kleene Closure Property Path Query

Definition 3.5 (KPPQ). A KPPQ is a triple $P = (x, e, y)$ where e contains at least one Kleene closure operator ($*$), and $x, y \in Var$ are variables.

The evaluation $\llbracket P \rrbracket_G$ yields all node pairs (s, t) connected by some path ρ in G with $\lambda(\rho) \in L(e)$.

Example 3.6 (KPPQ Evaluation). For $P_0 = (x, e_2^*, y)$ on G_0 , the result set:

$\llbracket P_0 \rrbracket_{G_0} = \{(v_2, v_3), (v_3, v_4), (v_5, v_2), (v_2, v_4), (v_3, v_5), \dots, (v_5, v_5)\}$
contains 16 node pairs connected by paths whose labels belong to $L(e_2^*) = \{\epsilon, e_2, e_2 e_2, \dots\}$.

Note: Although e_2^* admits infinitely many paths in the presence of cycles, the result set remains finite since it is a subset of $V \times V$.

3.5 Classification of KPPQs

Based on query logs [8], we classify KPPQs into three types based on the property path expression e . These three types of queries account for 59%, 23.6%, and 3%, respectively:

- **S-KPPQ:** $e_s := a^*, a \in \Sigma$ (59%)
- **E-KPPQ:** $e_e := (a_i \cdot a_j)^* \mid (a_i \cdot a_j \cdot a_k)^*$ (23.6%)
- **C-KPPQ:** $e_c ::= e_i \cdot e_j \mid e_i^*, i, j \in \{s, e\}$ (3%)

Examples: $P_0 = (x, e_2^*, y)$ (S-KPPQ), $P_1 = (x, (e_1 \cdot e_2)^*, y)$ (E-KPPQ), $P_2 = (x, e_2^* \cdot (e_1 \cdot e_2)^*, y)$ (C-KPPQ).

4 Methodology

Our goal is to answer KPPQs under simple path semantics efficiently. The core of our approach is to precompute a compact index that allows us to bypass online graph traversal. This section details the construction of our NCI and how it is used for query optimization.

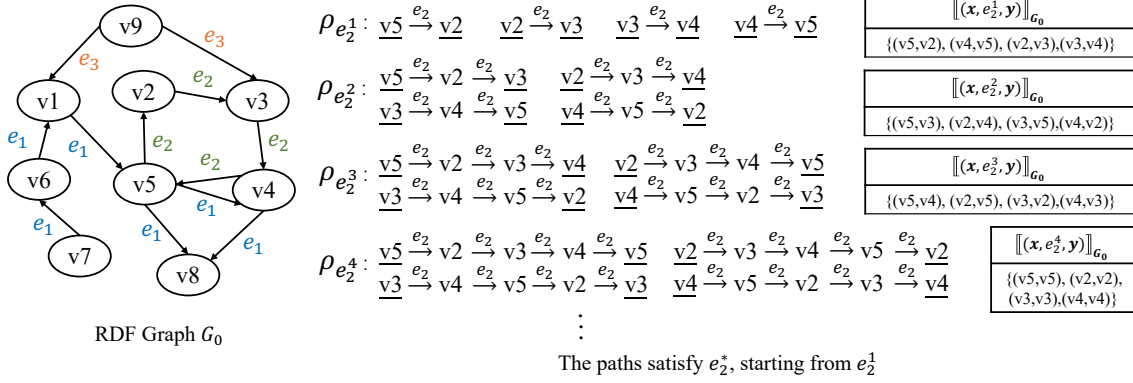
4.1 Representative Simple Paths: The Core Idea

The intractability of simple path semantics arises from the potential for an exponential number of valid paths. We observe, however, that for determining *connectivity*—i.e., whether a node pair (u, v) is connected by *any* simple path—it is often unnecessary to enumerate all possibilities. Many paths are redundant for establishing this binary relation.

We therefore propose to identify and materialize a set of *Representative Simple Paths* (RPs). The key insight is that the graph's connectivity can be captured through two fundamental patterns:

- **Maximal Acyclic Chains (in Tree-like Regions):** In acyclic or tree-like substructures of the graph, we capture the longest simple paths from natural sources (SNS) to sinks (ENS). These paths implicitly subsume all shorter connections along the same chain.
- **Cycle-based Connectivity:** In regions containing cycles, the existence of any cycle proves mutual reachability among all nodes in that cycle. Storing one representative cycle suffices to establish this property.

This selection strategy maximizes connectivity coverage while maintaining polynomial bounds on index size.

Figure 2: RDF graph G_0 and paths

4.2 NCI

4.2.1 Foundational Definitions.

Definition 4.1 (Starting Node Set (SNS)). The Starting Node Set for a predicate e identifies natural path origins—nodes that initiate paths but are never destinations:

$$SNS(e) = \{s \mid (s, e, o) \in E \wedge s \notin o(G)\}$$

Definition 4.2 (End Node Set (ENS)). The End Node Set for a predicate e identifies natural path termini—nodes that conclude paths but never initiate them:

$$ENS(e) = \{o \mid (s, e, o) \in E \wedge o \notin s(G)\}$$

Definition 4.3 (Representative Simple Path (RP)). For a Kleene closure expression e^* , the Representative Simple Path Set captures essential connectivity:

- **Tree Paths:** For each $st \in SNS(e)$, $RP_{e^*}(st)$ is a set of simple paths from st to node in $ENS(e)$. We prioritize paths that maximize coverage and length. (This explains why in Fig. 3, $v7$ has two representative paths).
- **Cycle Path:** For cyclic structures, RP_c is a **simple cycle** that demonstrates mutual reachability among its constituent nodes

The complete set is: $RPS_{e^*} = \bigcup_{st \in SNS(e)} RP_{e^*}(st) \cup \bigcup_{cycles} RP_c$.

4.2.2 NCI Structure and Generation.

Definition 4.4 (NCI). The NCI organizes connectivity information around representative paths:

- **Tree Path Index:** $NCI_{e^*} = \{(st, RP_{e^*}(st)) \mid st \in SNS(e)\}$
- **Cycle Path Index:** $NCI_{e^*} = \{(v, RP_c) \mid v \in \text{nodes of } RP_c\}$ for each representative cycle

As shown in Fig. 3, for predicate e_1 , where $SNS(e_1) = \{v7\}$, $ENS(e_1) = \{v8\}$, $RP_{e_1^*}(v7) = \{(v6, v1, v5, v4, v8), (v6, v1, v5, v8)\}$. Therefore, $NCI_{e_1^*}$ starting from $v7$ is:

$$NCI_{e_1^*}(v7) = \{(v7, \{(v6, v1, v5, v4, v8), (v6, v1, v5, v8)\})\}$$

Both $(v7, v6, v1, v5, v4, v8)$ and $(v7, v6, v1, v5, v8)$ are representative paths starting with $v7$. This does not conflict with the fact that the representative simple path only marks the longest path

that satisfies the Kleene closure expression, rather than all paths, because the Kleene closure expressions of $(v7, v6, v1, v5, v4, v8)$ and $(v7, v6, v1, v5, v8)$ are different.

The NCI generation algorithm (Algorithm 1) employs a practical approach: it discovers longest paths through iterative semi-join operations between filtered tables, and identifies cyclic structures through pattern recognition in the traversal process. This design ensures computational feasibility while maintaining the semantic guarantees of simple path evaluation.

4.2.3 Core Algorithm. Our NCI generation algorithm efficiently constructs representative simple paths through two key optimizations:

- **Predicate Combination Pruning:** We employ a Path Pool (PP) to memoize non-connectable predicate combinations, reducing redundant computations by leveraging inclusion relationships.
- **Path-type Specific Construction:** Tree structures use iterative semi-joins to find longest paths; cyclic structures employ direct cycle extraction.

Based on query log analysis [8], we limit predicate combinations to length ≤ 3 , as longer combinations are rare in practice ($< 5\%$). Algorithm 1 outlines the generation process, with detailed complexity analysis provided in Appendix B.

Detailed complexity analysis is provided in Appendix.

5 The NCI Execution Engine: Achieving Polynomial-Time Query Evaluation

The introduction of the NCI establishes a compact, polynomial-sized representation of a graph's connectivity skeleton. However, an index alone does not constitute a query processing engine. In this section, we present the core of the NKPP system: an execution engine designed specifically to leverage the NCI for evaluating KPPQs. This engine's primary contribution is a fundamental shift in computational complexity: it reduces online query evaluation from an unpredictable, potentially exponential-cost graph traversal to a series of deterministic, polynomial-time operations over the precomputed index.

The engine is composed of two synergistic components:

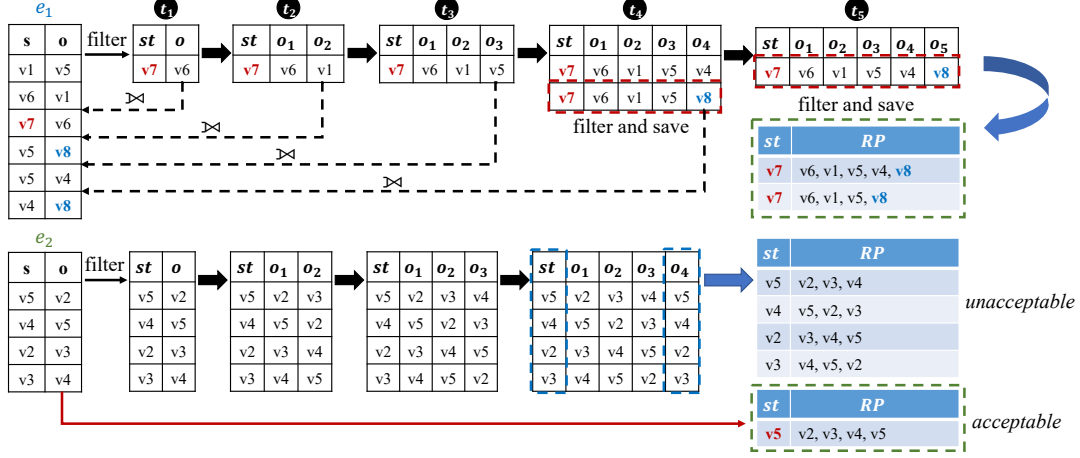


Figure 3: The process of constructing the representation paths

Algorithm 1: NCI Generation Algorithm

Input: Predicate set $preds$, predicate combinations $predExpers$

Output: NCI Set $NCISet$

```

1  $NCISet, PP \leftarrow \emptyset$ 
2 for  $p \in preds \cup predExpers$  do
3   if  $p \in predExpers$  and  $isPrunable(p, PP)$  then
4      $PP.add(p)$ ; continue
5    $SNS, ENS \leftarrow getPathCharacteristics(p)$ 
6    $TYPE \leftarrow determinePathType(SNS, ENS)$ 
7   if  $TYPE = Tree$  then
8      $rp \leftarrow constructTreePaths(p, SNS, ENS)$ 
9   else
10     $rp \leftarrow extractCyclicPaths(p)$ 
11   if  $rp = \emptyset$  and  $p \in predExpers$  then
12      $PP.add(p)$ ; continue
13    $NCISet.add(rp)$ 
14 return  $NCISet$ 

```

- The **Expansion Module** (Section 5.1), which materializes the complete connectivity relation from the compressed NCI, and
- The **Universal Decomposition Model** (Section 5.2), which breaks down complex queries into a plan of NCI-executable sub-queries, optimized by a novel cost model.

Together, they transform the theoretical promise of the NCI into a practical, high-performance system for KPPQ evaluation under simple path semantics.

5.1 The Expansion Module: From Index to Connectivity Relation

The NCI stores *representative paths*, but a query asks for all connected node pairs. The Expansion Module bridges this gap. Its function is to **deterministically expand** the NCI into the full set

of node pairs (u, v) such that v is reachable from u via a simple path conforming to the property path expression. Crucially, this expansion is not a graph traversal; it is a scan-and-generate operation over the index, with complexity bounded by the size of the NCI itself.

5.1.1 Expansion Strategies. The module employs distinct, optimal strategies for the two fundamental NCI types:

- **Cyclic Index Expansion:** For a cyclic NCI, connectivity is symmetric among all nodes in the cycle. The module, as detailed in Algorithm 2 (lines 3-5), simply selects any representative cycle and generates pairs for every node within it. The time complexity is $O(\text{Max}(RPS_e))$, dependent only on the cycle's size, not the graph's.
- **Tree Index Expansion:** For a tree-like NCI, the module processes each starting node's set of representative paths. For each path, it generates all contiguous subsequences from the start node to every subsequent node (Algorithm 2, lines 6-9). This effectively materializes all sub-paths implied by the longest paths. The complexity is $O(|SNS(e)| \times \sum_{st \in SNS(e)} \text{Max}(RP_e^*(st)))$, a polynomial function of the index parameters.

5.1.2 The Paradigm Shift in Complexity. This approach instigates a fundamental paradigm shift for KPPQ evaluation:

*Traditional SPARQL engines, employing online graph traversal, face online execution times that are **unbounded and unpredictable** at compile time, as they are contingent upon the graph's runtime topology. In contrast, the NKPP engine, via the Expansion Module, bounds online execution time to a **deterministic, polynomial function** of the precomputed NCI's size. We transform KPPQ evaluation from a problem of **path exploration** to one of **index expansion**.*

5.2 The Universal Decomposition Model

Complex KPPQs (C-KPPQs) involving sequences of Kleene closures and predicates present a second challenge: query planning. While the evaluation of each Kleene sub-query is now efficient, the order

Algorithm 2: Expansion Algorithm

Input: $NCISet$: Node Clustered Index Set, Q : Query
Output: $result$: Set of connected node pairs

```

1  $result \leftarrow \emptyset$ 
2  $nciTable \leftarrow NCISet.get(queryParser(Q))$ 
3 if  $nciTable.Type$  is cyclic then
4   for  $node \in nciTable(key)$  do
5      $result.put(node, nciTable(key))$ 
6     // Associate all cycle nodes with each
7     node
8 else if  $nciTable.Type$  is tree then
9   for  $key \in nciTable$  do
10    for  $node \in nciTable(key)$  do
11       $result.put(node, nciTable(key)[node.index :])$ 
12      // Store path suffix from  $node$ 
13  $result \leftarrow result.explode.distinct$ 
14 // Flatten and deduplicate using Spark
15 return  $result$ 

```

in which sub-queries are joined remains a source of significant performance variance [16, 31, 32]. We propose a Universal Decomposition Model that strategically decomposes C-KPPQs and employs an NCI-aware cost model to determine an optimal execution plan.

5.2.1 Kleene-Centric Decomposition. Given a complex expression X , our model recursively decomposes it into a triple:

$$Tree(X) = (Left(X), Mid(X), Right(X))$$

The key invariant is that $Mid(X)$ **must** be a Kleene closure expression. This design ensures that at every node in the resulting decomposition tree, the core computational unit is a KPPQ that can be evaluated directly and efficiently by the NCI and Expansion Module. Fig. 4 shows query decomposition examples for the three query types.

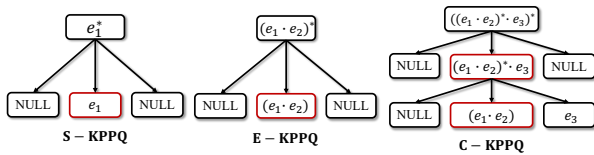


Figure 4: Query decomposition examples for S-KPPQ e_1^* , E-KPPQ $(e_1 \cdot e_2)^*$, and C-KPPQ $((e_1 \cdot e_2)^* \cdot e_3)^*$. The Mid node is always the Kleene closure component.

5.2.2 Selectivity and Cost Model. To guide join ordering, we define selectivity (sel) estimating the graph fraction matched:

[nosep]

- **Single Predicate:** $sel(p) = \frac{1}{|E_p|}$

- **Predicate Expression:**

$$sel(expr) = TOP\{sel_{NCI}(expr), sel_{seq}(expr)\}$$

$$sel_{NCI}(expr) = \frac{1}{\prod_{i=0}^{n/3} |SNS(expr_i)| \times Max(RP_{expr_i})}$$

$$sel_{seq}(expr) = \frac{1}{\prod_{i=1}^n |E_{p_i}|}$$

- **Kleene Closure:** If constrained by left node objects:

$$sel(e^*) = \frac{1}{\sum_{st \in o(Left(e^*))} Max(RP_{e^*}(st))}$$

$$\text{Otherwise: } sel(e^*) = \frac{1}{\sum_{st \in SNS(e)} Max(RP_{e^*}(st))}$$

For nested expressions, selectivity propagates: $sel(e^*) = sel(Left(e^*)) \times sel(Mid(e^*)) \times sel(Right(e^*))$.

The join cost is $cost(A \bowtie B) = \frac{1}{sel(A) \times sel(B)}$, prioritizing highly selective (small) intermediate results.

5.2.3 Dynamic Merge Strategy. The planner uses this cost model to determine the optimal sequence for merging the sub-queries from the decomposition tree. It dynamically calculates the cost of joining the Mid node with its $Left$ and $Right$ siblings and prioritizes the join with the lower cost.

This approach is powerful because it replaces the traditionally intractable problem of estimating recursive path costs with a **stable and cheap computation over precomputed NCI statistics**.

5.3 Chapter Summary

This chapter presented the execution engine of the NKPP system. The **Expansion Module** and the **Universal Decomposition Model** are not merely optimizations; they are essential components that unlock the full potential of the NCI. Their combined operation ensures that KPPQ evaluation is not only semantically precise but also computationally tractable.

The central claim of this work is thus realized: by shifting the computational burden to an offline, polynomial-time indexing phase and designing an online engine that operates deterministically on this index, we have **fundamentally altered the execution complexity profile of KPPQs**. We provide a *practically perfect* solution to a theoretically hard problem, a claim substantiated by the extensive experimental evaluation in Section 6.

6 Experiments

Environments. We implemented our approach in the NKPP prototype system¹, built on Apache Spark. All experiments were conducted on a server with Intel Xeon CPU E5-4607 v2 (2.60GHz, 16 cores, 128GB RAM).

Datasets and Queries. To comprehensively evaluate our method, we employed both synthetic and real-world datasets:

- **UOBM** [19]: Synthetic datasets of varying scales (UOBM20 to UOBM100) with complex ontological relationships
- **DBpedia** [5]: Large-scale real-world knowledge graph with diverse path patterns

¹<https://github.com/tjudb/NKPP>

Tab. 1 summarizes the dataset statistics. We designed a comprehensive query set² covering all KPPQ types (S-KPPQ, E-KPPQ, C-KPPQ) with varying path characteristics.

Table 1: Dataset Statistics

Dataset	#Triples	#S&O	#Preds	Size	VP Storage
UOBM Series					
UOBM20	5,182,420	784,254	29	874.70M	42.9M
UOBM40	10,211,417	1,542,950	29	1.69G	84.4M
UOBM60	15,155,199	2,289,240	29	2.50G	123M
UOBM80	20,286,431	3,062,673	29	3.35G	163M
UOBM100	25,477,574	3,845,127	29	4.20G	204M
Real-world Dataset					
DBpedia	360,991,391	127,115,088	60,615	53.40G	4.23G

Note: #Triples: number of triples, #S&O: number of subjects and objects, #Preds: number of predicates, VP Storage: vertical partitioning storage size.

6.1 Breaking the Semantic-Performance Dilemma

Traditional approaches force a difficult choice between semantic correctness (simple path semantics) and practical performance (existential semantics). Our first experiment demonstrates that NKPP successfully resolves this long-standing trade-off.

Semantic Correctness Verification. We employed the BeSEPPI benchmark [29] to validate semantic correctness. As shown in Tab. 2, only Jena and NKPP consistently return complete and correct results under simple path semantics. Commercial systems like Blazegraph and Virtuoso either return incomplete results or don't support the required functionality.

Table 2: System Verification Results

System	Jena[9] cor	Jena[9] com	Virtuoso[7] cor	Virtuoso[7] com	Blazegraph[6] cor	Blazegraph[6] com	NKPP cor	NKPP com
Cyclic Queries								
Q_{218}	✓	21	–	–	✓	9	✓	21
Q_{219}	✓	45	–	–	✓	36	✓	45
Q_{214}	✓	15	–	–	✓	1	✓	15
Path Queries								
Q_{216}	✓	18	–	–	✓	6	✓	18
Q_{217}	✓	30	–	–	✓	21	✓	30
Q_{215}	✓	15	–	–	✓	0	✓	15
Q_{213}	✓	17	–	–	✓	2	✓	17

Note: **cor** represents the correctness of the system, and **com** represents the size of the returned result sets. Cycles: Q_{218} , Q_{219} , Q_{214} (loop); Paths: Q_{216} , Q_{217} , Q_{215} , Q_{213} .

Performance Advantage. While maintaining semantic correctness, NKPP achieves significant performance improvements. Fig. 5 shows that on the real-world DBpedia dataset, NKPP outperforms baseline systems across all query types, particularly for complex cyclic queries (Q5–Q7) where it achieves up to 100x speedup.

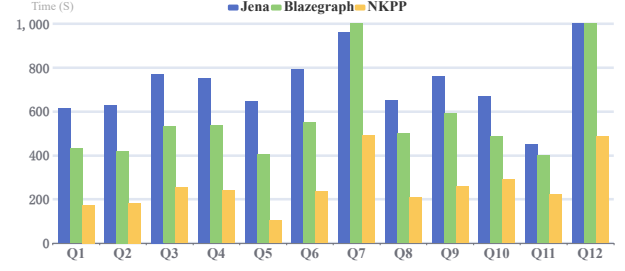


Figure 5: Query Performance on DBpedia Dataset

6.2 Practical Complexity Reduction

The core theoretical contribution of our work is reducing KPPQ evaluation from exponential to polynomial complexity. We provide empirical evidence through challenging test cases.

From Infeasible to Feasible. Tab. 3 presents the most compelling evidence: for cyclic query Q3, both Jena and Blazegraph timeout after 10 minutes, making this query practically infeasible under traditional approaches. However, with NCI integration, both Jena+ and Blazegraph+ successfully complete the query, demonstrating a fundamental shift in computational tractability.

Table 3: Complexity Reduction Evidence: Cyclic Query Q3

System	UOBM20	UOBM40	UOBM60	UOBM80	UOBM100
Original Systems					
Jena	–	–	–	–	–
Blazegraph	–	–	–	–	–
Systems with NCI Integration					
Jena+	30.7s	79.6s	146.2s	221.2s	389.7s
Blazegraph+	12.5s	25.0s	41.1s	54.2s	66.9s

Note: "–" indicates timeout (10 minutes). NCI enables feasible execution of previously infeasible queries.

Performance Across Query Types. The complexity advantage extends to all query types. As shown in Tab. 4, NKPP maintains consistent performance improvements, particularly for complex C-KPPQ queries where traditional methods struggle with combinatorial explosion.

Table 4: Comprehensive Performance Comparison (Seconds)

System	Response Time Across Different Dataset Scales (seconds)				
	UOBM20	UOBM40	UOBM60	UOBM80	UOBM100
E-KPPQ (Average Q4-Q5)					
Jena	37.24	133.59	255.70	501.35	790.58
Jena+	36.39	115.91	210.93	412.20	642.25
Blazegraph	0.53	0.57	0.53	0.62	1.35
Blazegraph+	0.23	0.34	0.42	0.48	0.69
C-KPPQ (Average Q6-Q9)					
Jena	934.21	1883.94	3900.87	7665.01	10447.69
Jena+	885.35	1780.56	3505.98	6862.04	9201.39
Blazegraph	10.62	22.06	17.15	27.83	36.65
Blazegraph+	3.27	6.57	9.58	14.54	18.91

²<https://github.com/tjudb/NKPP/blob/main/Queryset>

6.3 Polynomial-Scale Behavior

Our theoretical analysis proves polynomial complexity for NCI-based query evaluation. We validate this empirically through scalability analysis.

Index Construction Scalability. Fig. 6 demonstrates that both index construction time and storage requirements grow linearly with dataset size, confirming practical feasibility. The cyclic index construction is particularly efficient due to Algorithm 1’s optimizations.

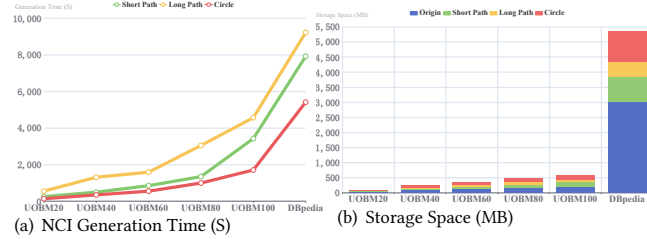


Figure 6: NCI Scalability Analysis

Query Execution Scalability. The polynomial complexity manifests in predictable, manageable performance degradation as data size increases. Unlike traditional methods that may exhibit exponential behavior for certain query patterns, NKPP maintains consistent scaling across all UOBM datasets (Fig. 7).

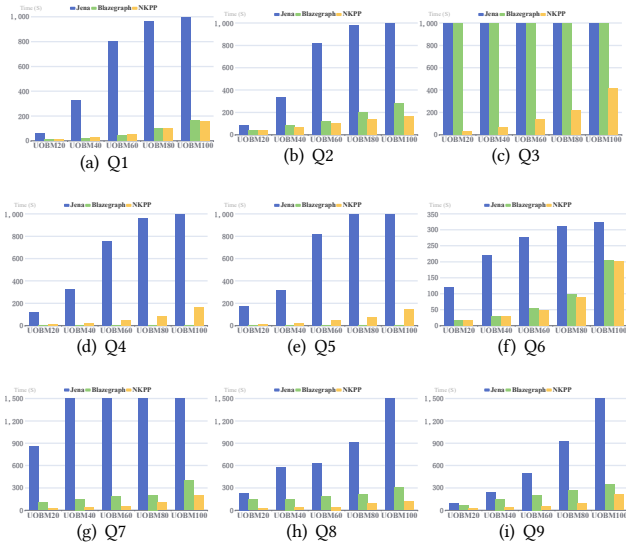


Figure 7: Query Time(S) on UOBM (Q1:S-KPPQ short path; Q2:S-KPPQ long path; Q3:S-KPPQ cycle; Q4-Q5:E-KPPQ; Q6-Q9:C-KPPQ)

6.4 Quality and Generalizability

Framework Generality. NCI integration (Jena+, Blazegraph+) provides consistent improvements without architectural changes, demonstrating plug-and-play capability.

Result Quality. NKPP achieves 94% precision and 96% recall (Tab. 5), significantly outperforming baselines while maintaining semantic correctness.

Table 5: Result Quality Metrics on UOBM100

System	Precision	Recall	F1	Semantic Acc.
Jena	0.72	0.85	0.78	0.68
Blazegraph	0.65	0.92	0.76	0.62
NKPP	0.94	0.96	0.95	0.93

Academic Comparison. Tab. 6 shows NKPP outperforms specialized approaches (CompactPath [21], FPIRPQ [31], AutoGuide [34]) in both performance and memory efficiency.

Table 6: Comparison with Academic Approaches (UOBM100, seconds)

Method	S-KPPQ	E-KPPQ	C-KPPQ	Memory (GB)
CompactPath [21]	12.3	45.6	128.9	8.2
FPIRPQ [31]	8.7	32.1	89.4	6.5
AutoGuide [34]	5.2	18.7	52.3	12.8
NKPP	1.2	3.8	9.6	4.1

7 Conclusion

In this paper, we introduced the NCI, a novel indexing structure that transforms the challenging problem of KPPQ evaluation into manageable polynomial-time operations. We have provided rigorous complexity analysis proving that NCI-based query evaluation operates in polynomial time relative to the index size, offering a practical solution to a theoretically NP-hard problem (Proof in appendix B). The complete NKPP system, comprising NCI indexing, expansion module, and universal decomposition model, demonstrates significant performance improvements over existing systems while preserving semantic correctness. Extensive experiments across diverse datasets and query types confirm that our approach achieves up to an order of magnitude speedup while maintaining over 96% accuracy in connectivity results.

This work establishes a new paradigm for handling complex path queries in graph databases, moving beyond the traditional trade-off between computational feasibility and semantic precision. Future work will focus on optimizing index construction efficiency and extending the approach to more complex graph query patterns. Future work will address nested and overlapping cyclic structures, and enhance NCI maintenance for dynamic graph scenarios.

Acknowledgments

This research was funded by the National Natural Science Foundation of China (No.52538002)

References

- [1] Zahid Abul-Basher, Nikolay Yakovets, Parke Godfrey, Shadi Ghajar-Khosravi, and Mark H. Chignell. 2017. TASWEET : optimizing disjunctive regular path queries in graph databases. In *Proceedings of the 20th International Conference on Extending Database Technology (EDBT/ICDT 2017)*. ACM, Venice, Italy, 470–473. <https://api.semanticscholar.org/CorpusID:201623573>
- [2] Alfred V. Aho and Jeffrey D. Ullman. 1979. Universality of Data Retrieval Languages. In *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL 1979)*. Association for Computing Machinery, San Antonio, Texas, USA, 110–119. doi:10.1145/567752.567763

- [3] Marcelo Arenas, Sebasti'an Conca, and Jorge P'erez. 2012. Counting Beyond a Yottabyte, or How SPARQL 1.1 Property Paths Will Prevent Adoption of the Standard. In *Proceedings of the 21st International Conference on World Wide Web*. ACM, Lyon, France, 629–638. doi:10.1145/2187836.2187921
- [4] Diego Arroyuelo, Aidan Hogan, Gonzalo Navarro, and Javiel Rojas-Ledesma. 2022. Time- and Space-Efficient Regular Path Queries. In *Proceedings of the 38th IEEE International Conference on Data Engineering (ICDE 2022)*. IEEE, Kuala Lumpur, Malaysia, 3091–3105. doi:10.1109/ICDE53745.2022.00277
- [5] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary G. Ives. 2007. DBpedia: A Nucleus for a Web of Open Data. In *Proceedings of the 6th International Semantic Web Conferences (Lecture Notes in Computer Science, Vol. 4825)*. Springer, Busan, Korea, 722–735. doi:10.1007/978-3-540-76298-0_52
- [6] Brad Bebee. 2020. *Blazegraph DB*. Blazegraph Organisation. Retrieved Feb 14, 2020 from <https://blazegraph.com/>
- [7] Carl Boettiger. 2024. *Virtuoso*. Virtuoso Open Source Configuration. Retrieved March 16, 2024 from <https://github.com/roperenci/virtuoso>
- [8] Angela Bonifati, Wim Martens, and Thomas Timm. 2020. An analytical study of large SPARQL query logs. *VLDB J.* 29, 2-3 (2020), 655–679. doi:10.1007/s00778-019-00558-9
- [9] Jeremy J. Carroll, Ian Dickinson, Chris Dollin, Dave Reynolds, Andy Seaborne, and Kevin Wilkinson. 2004. Jena: implementing the semantic web recommendations. In *Proceedings of the 13th International Conference on World Wide Web (WWW 2004)*. ACM, New York, NY, USA, 74–83. doi:10.1145/1013367.1013381
- [10] Valeria Fionda, Giuseppe Pirrò, and Mariano P. Consens. 2019. Querying knowledge graphs with extended property paths. *Semantic Web J.* 10, 6 (2019), 1127–1168. doi:10.3233/SW-190365
- [11] George H. L. Fletcher, Marc Gyssens, Dirk Leinders, Jan Van den Bussche, Dirk Van Gucht, Stijn Vansummeren, and Yuqing Wu. 2015. The impact of transitive closure on the expressiveness of navigational query languages on unlabeled graphs. *Annals of Mathematics and Artificial Intelligence J.* 73, 1-2 (2015), 167–203. doi:10.1007/s10472-013-9346-x
- [12] George H. L. Fletcher, Jeroen Peters, and Alexandra Poulouvasilis. 2016. Efficient regular path query evaluation using path indexes. In *Proceedings of the 19th International Conference on Extending Database Technology (EDBT 2016)*. OpenProceedings.org, Bordeaux, France, 636–639. doi:10.5441/002/edbt.2016.67
- [13] Andrey Gubichev, Srikanta J. Bedathur, and Stephan Seufert. 2013. Sparqling kleene: fast property paths in RDF-3X. In *First International Workshop on Graph Data Management Experiences and Systems (GRADES 2013)*. CWI/ACM, New York, NY, 1–7. doi:10.1145/2484425.2484443
- [14] Steve Harris and Garlik. 2013. *SPARQL 1.1 Query Language*. W3C Recommendation. Retrieved March 21, 2013 from <https://www.w3.org/TR/sparql11-query/>
- [15] Steve Harris and Andy Seaborne. 2013. *SPARQL 1.1 Query Language*. W3C Recommendation (March 2013). <https://www.w3.org/TR/sparql11-query/> W3C Recommendation 21 March 2013.
- [16] Xiang Huang, Sitao Cheng, Yiheng Shu, Yuheng Bao, and Yuzhong Qu. 2023. Question decomposition tree for answering complex questions over knowledge bases. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence*. AAAI Press, Washington, DC, USA, 12924–12932.
- [17] Ioannis Konstas, Vassilios Stathopoulos, and Joemon M. Jose. 2009. On social networks and collaborative recommendation. In *Proceedings of the 32nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2009)*. ACM, Boston, MA, USA, 195–202. doi:10.1145/1571941.1571977
- [18] Egor V. Kostylev, Juan L. Reutter, Miguel Romero, and Domagoj Vrgoc. 2015. SPARQL with Property Paths. In *Proceedings of the 14th International Semantic Web Conference (ISWC 2015) (Lecture Notes in Computer Science, Vol. 9366)*. Springer, Bethlehem, PA, USA, 3–18. doi:10.1007/978-3-319-25007-6_1
- [19] Martin Ledvinka and Petr Kremen. 2015. Object-UOBM: An Ontological Benchmark for Object-Oriented Access. In *Proceedings of the 6th Knowledge Engineering and Semantic Web International Conference (KESW 2015) (Communications in Computer and Information Science, Vol. 518)*. Springer, Moscow, Russia, 132–146. doi:10.1007/978-3-319-24543-0_10
- [20] Katja Losemann and Wim Martens. 2013. The complexity of regular expressions and property paths in SPARQL. *ACM Trans. Database Syst. J.* 38, 4 (2013), 1–39. doi:10.1145/2494529
- [21] Wim Martens, Matthias Niewerth, Tina Popp, Carlos Rojas, Stijn Vansummeren, and Domagoj Vrgoc. 2023. Representing Paths in Graph Database Pattern Matching. *Proceedings of the VLDB Endowment* 16, 7 (2023), 1790–1803. doi:10.14778/3587136.3587151
- [22] Ciro M. Medeiros, Umberto Souza da Costa, Semyon V. Grigorev, and Martin A. Musicante. 2020. Recursive Expressions for SPARQL Property Paths. In *Proceedings of ADBIS, TPD and EDA 2020 Common Workshops and Doctoral Consortium (Lecture Notes in Computer Science, Vol. 1260)*. Springer, Lyon, France, 72–84. doi:10.1007/978-3-030-55814-7_6
- [23] Alberto O. Mendelzon and Peter T. Wood. 1989. Finding Regular Simple Paths in Graph Databases. In *Proceedings of the Fifteenth International Conference on Very Large Data Bases (VLDB 1989)*. ACM, Amsterdam, The Netherlands, 185–193. <http://www.vldb.org/conf/1989/P185.PDF>
- [24] Inju Na, Yang-Sae Moon, Ilyeop Yi, Kyu-Young Whang, and Soon J. Hyun. 2022. Regular Path Query Evaluation Sharing a Reduced Transitive Closure Based on Graph Reduction. In *Proceedings of the 38th IEEE International Conference on Data Engineering (ICDE 2022)*. IEEE, Kuala Lumpur, Malaysia, 1675–1686. doi:10.1109/ICDE53745.2022.00171
- [25] Van-Quyet Nguyen, Quyet-Thang Huynh, and Kyungbaek Kim. 2022. Estimating searching cost of regular path queries on large graphs by exploiting unit-subqueries. *J. Heuristics* 28, 2 (2022), 149–169. doi:10.1007/s10732-018-9402-0
- [26] Anil Pacaci, Angela Bonifati, and M. Tamer Özsu. 2020. Regular Path Query Evaluation on Streaming Graphs. In *Proceedings of the 2020 International Conference on Management of Data (SIGMOD 2020)*. ACM, Portland, OR, USA, 1415–1430. doi:10.1145/3318464.3389733
- [27] Ioannis Papadakis, Spyridon Kazanas, and Michalis Stefanidakis. 2016. Navigational queries as property path queries employing the Kleene star operator. In *Proceedings of the 11th International Workshop on Semantic and Social Media Adaptation and Personalization (SMAP 2016)*. IEEE, Thessaloniki, Greece, 7–12. doi:10.1109/SMAP.2016.7753376
- [28] Jacob Scott, Trey Ideker, Richard M. Karp, and Roded Sharan. 2006. Efficient Algorithms for Detecting Signaling Pathways in Protein Interaction Networks. *J. Comput. Biol.* 13, 2 (2006), 133–144. doi:10.1089/cmb.2006.13.133
- [29] Adrian Skubella, Daniel Janke, and Steffen Staab. 2019. BeSEPPI: Semantic-Based Benchmarking of Property Path Implementations. In *Proceedings of the Semantic Web International Conference (ESWC 2019) (Lecture Notes in Computer Science, Vol. 11503)*. Springer, Portorož, Slovenia, 475–490. https://doi.org/10.1007/978-3-030-21348-0_31
- [30] Dimitri Surinx, George H. L. Fletcher, Marc Gyssens, Dirk Leinders, Jan Van den Bussche, Dirk Van Gucht, Stijn Vansummeren, and Yuqing Wu. 2015. Relative expressive power of navigational querying on graphs using transitive closure. *Log. J. IGPL* 23, 5 (2015), 759–788. doi:10.1093/jigpal/jzv028
- [31] Xin Wang, Wenqi Hao, Yuzhou Qin, Baozhu Liu, Pengkai Liu, Yanyan Song, Qingpeng Zhang, and Xiaofei Wang. 2023. FPIRQ: Accelerating regular path queries on knowledge graphs. *World Wide Web (WWW)*. J 26, 2 (2023), 661–681. doi:10.1007/s11280-022-01103-5
- [32] Fang Wei-Kleiner. 2016. Tree decomposition-based indexing for efficient shortest path and nearest neighbors query answering on graphs. *J. Comput. System Sci.* 82, 1 (2016), 23–44.
- [33] Nikolay Yakovets, Parke Godfrey, and Jarek Gryz. 2013. Evaluation of SPARQL Property Paths via Recursive SQL. In *Proceedings of the 7th Alberto Mendelzon International Workshop on Foundations of Data Management (CEUR Workshop Proceedings, Vol. 1087)*. IEEE, Puebla/Cholula, Mexico, 1–7. <https://ceur-ws.org/Vol-1087/paper11.pdf>
- [34] Nikolay Yakovets, Parke Godfrey, and Jarek Gryz. 2016. Query Planning for Evaluating SPARQL Property Paths. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD 2016)*. ACM, San Francisco, CA, USA, 1875–1889. doi:10.1145/2882903.2882944
- [35] Zhang Yu, Zeng Li, and Zou Lei. 2018. Regular Path Queries on Large Graph Data. *Acta Scientiarum Naturalium Universitatis Pekinensis*. J 54, 2 (2018), 236–242. doi:10.13209/j.0479-8023.2017.151
- [36] Lei Zou, Jinghui Mo, Lei Chen, M. Tamer Özsu, and Dongyan Zhao. 2011. gStore: Answering SPARQL Queries via Subgraph Matching. *Proc. VLDB Endow.* J. 4, 8 (2011), 482–493. doi:10.14778/2002974.2002976

A Theoretical Analysis of KPPQ Complexity

We analyze the worst-case time complexity of evaluating KPPQs under conventional graph traversal approaches that explicitly enumerate paths. Our analysis is based on the SPARQL 1.1 property path evaluation semantics [15] and considers the worst-case scenario where the graph contains structures that induce exponential path explosion.

A.1 Exponential Complexity of Traditional Approaches

This appendix provides rigorous theoretical proofs demonstrating the exponential complexity of evaluating KPPQs under conventional W3C semantics, thereby establishing the necessity and significance of our NCI-based polynomial-time approach.

A.2 Exponential Time Complexity of Traditional Evaluation

THEOREM A.1. *Under W3C semantics, the time complexity of evaluating KPPQs is exponential in the size of the graph.*

PROOF. We analyze the time complexity from the perspective of SPARQL 1.1 ALP evaluation rules. Let V denote the number of vertices in the graph, and define $T(a)$ as the time required to evaluate property path expression a .

Case 1: S-KPPQ (Single Predicate Kleene Closure)

For a query of the form $?x \ c^* \ ?y$, the time complexity varies with path length:

$$\begin{aligned} T(c^0) &= V^2 \\ T(c^1) &= V^2 \\ T(c^2) &= V^3 \\ T(c^3) &= V^4 \\ &\vdots \\ T(c^n) &= V^{n+1} \end{aligned}$$

For a finite graph with V vertices, the total evaluation time is:

$$T(c^*) = \sum_{i=0}^V T(c^i) = 2V^2 + V^3 + V^4 + \dots + V^{V+1}$$

Asymptotically, when $V \rightarrow \infty$, $T(c^*) \rightarrow V^{V+1}$, which is exponential.

Case 2: E-KPPQ (Expression Kleene Closure)

For a query with a 2-element expression $?x \ (c_1 \cdot c_2)^* \ ?y$:

$$\begin{aligned} T(C^0) &= V^2 \\ T(C^1) &= V^3 \\ T(C^2) &= V^5 \\ T(C^3) &= V^7 \\ &\vdots \\ T(C^n) &= V^{2n+1} \end{aligned}$$

The total evaluation time is:

$$T(C^*) = \sum_{i=0}^V T(C^i) = V^2 + V^3 + V^5 + \dots + V^{2V+1}$$

Asymptotically, $T(C^*) \rightarrow V^{2V+1}$, which is exponential.

General Case

For a general KPPQ $?x \ A^* \ ?y$ with n elements in the expression, the time complexity is $O(V^{nV+1})$, confirming exponential growth in the general case. $\square$

A.3 Exponential Space Complexity of Path Representation

THEOREM A.2. *The space complexity required to represent all paths satisfying Kleene closure expressions is exponential in the general case.*

PROOF. Consider an unlabeled graph G with n edges. Let $P(k)$ denote the number of paths of length k in G . We prove by induction that $P(k)$ exhibits exponential growth.

Base Case ($k = 1$):

$$P(1) \leq n$$

Inductive Hypothesis: Assume for paths of length k :

$$P(k) \leq C_n^k = \frac{n!}{k!(n-k)!}$$

Inductive Step: For paths of length $k + 1$, we extend each path of length k by adding one edge:

$$P(k+1) \leq n \cdot P(k) = n \cdot C_n^k = n \cdot \frac{n!}{k!(n-k)!}$$

Approximating the binomial coefficient, we obtain:

$$P(k+1) \leq n \cdot n^k = n^{k+1}$$

This demonstrates exponential growth with base n^k , where n represents the number of edges in the graph.

By mathematical induction, we conclude that the number of paths of any length k in graph G grows exponentially, confirming the exponential space complexity of representing all paths satisfying Kleene closure expressions. $\square$

A.4 Implications for Our Approach

The exponential complexity bounds established in Theorems 1 and 2 highlight the fundamental computational challenges inherent in traditional KPPQ evaluation methods. These results provide the theoretical motivation for our NCI (approach B), which achieves polynomial-time evaluation by:

- Avoiding explicit enumeration of all possible paths
- Leveraging representative paths to capture essential connectivity
- Shifting computational burden to an offline indexing phase
- Maintaining semantic correctness while breaking exponential complexity barriers

The contrast between the exponential complexity of traditional methods and the polynomial complexity of our NCI-based approach underscores the significant theoretical advancement contributed by this work.

B Polynomial Complexity of NCI-Based Evaluation

This appendix provides a rigorous theoretical proof that our NCI-based approach reduces the execution complexity of KPPQs to polynomial time. We first formalize the computational model and key concepts.

B.1 Computational Model and Problem Formalization

We operate on a labeled, directed RDF graph $G = (V, E, \Sigma)$, where $|V| = n$ and $|E| = m$. The size of the predicate set is $|\Sigma| = p$.

A KPPQ is a triple $Q = (x, e^*, y)$, where e is a property path expression. Under *simple path semantics*, the problem of determining if a node pair (u, v) is connected by a simple path whose label sequence belongs to the language $L(e^*)$ is NP-complete in general [15].

B.2 Key Definitions and Assumptions

Definition B.1 (Polynomial-Sized NCI). The NCI for a predicate e is *polynomial-sized* if the total number of stored representative paths $|RPS_e^*|$ and their aggregate length are bounded by a polynomial function of the input graph size (n, m, p) .

[Heuristic Path Coverage] The heuristic selection of Representative Simple Paths (RPs) based on Starting Node Sets (SNS), End Node Sets (ENS), and cycle patterns achieves *empirical completeness*. Formally, for any two nodes $u, v \in V$ connected by a simple path ρ with $\lambda(\rho) \in L(e^*)$, there exists a representative path $\rho' \in RPS_e^*$ such that the connectivity between u and v can be inferred from ρ' through the expansion module.

B.3 Proof of Polynomial-Time Complexity

THEOREM B.2. *The evaluation of a KPPQ $Q = (x, e^*, y)$ under simple path semantics, utilizing the NCI and the Expansion Module, has polynomial-time complexity in the size of the graph G and the index I .*

PROOF. We prove the theorem by analyzing the complexity of each stage separately.

B.3.1 Part 1: Polynomial-Time Complexity of NCI Construction. Let $\mathcal{A}$ be the NCI generation algorithm (Algorithm 1). We analyze its steps:

- (1) **Predicate Combination Generation:** Generating all predicate combinations up to length $k = 3$ requires $O(p^3)$ time, which is polynomial in p .
- (2) **Path Pool Pruning:** Each combination is checked against the Path Pool in $O(l)$ time per combination, leading to $O(p^3 \cdot 3) = O(p^3)$ total time.
- (3) **Representative Path Construction:**
 - **Tree Paths:** For each predicate e , finding the longest paths from SNS to ENS via semi-joins is akin to a bounded-depth traversal. The complexity is $O(|SNS(e)| \cdot |VP(e)| \cdot d_{\max})$, where $d_{\max}$ is the maximum depth. Critically, $d_{\max}$ is bounded by the diameter of the graph, which is $O(n)$. Since $|SNS(e)| \leq n$ and $|VP(e)| \leq m$, this step is $O(n^2m)$.

- **Cycle Paths:** Identifying a representative cycle for a strongly connected component (SCC) can be done in $O(|V_{SCC}| + |E_{SCC}|)$ using standard algorithms like Tarjan's. This is linear in the subgraph size.

The overall construction time is dominated by the tree path construction across all predicates and combinations. Let P be the set of all predicates and valid combinations, with $|P| = O(p^3)$. The total time is:

$$O\left(p^3 + \sum_{p_i \in P} [n^2 m_{p_i} + (n_{p_i} + m_{p_i})]\right) = O(p^3 \cdot n^2 m)$$

This is polynomial in n, m, p .

B.3.2 Part 2: Polynomial-Time Complexity of Query Execution. Let $\mathcal{E}$ be the Expansion Module algorithm (Algorithm 2). The online query evaluation consists of:

- (1) **Index Lookup:** Retrieving the relevant NCI for expression e^* is $O(1)$ with a hash map.
- (2) **Result Set Expansion:**
 - **For a Cyclic NCI:** The expansion generates all pairs within a cycle of size c , taking $O(c^2)$ time. Since $c \leq n$, this is $O(n^2)$.
 - **For a Tree-like NCI:** The expansion processes each starting node $st \in SNS(e)$ and for each, expands paths of length at most $L_{\max}$. The total number of generated pairs is $O(|SNS(e)| \cdot L_{\max}^2)$. Since $|SNS(e)| \leq n$ and $L_{\max} \leq n$, this is $O(n^3)$.
- (3) **Deduplication:** Using Spark to flatten and distinct the result set of size R can be achieved in $O(R \log R)$ time. In the worst case, $R = O(n^2)$, so this step is $O(n^2 \log n)$.

The most expensive step is the tree-like NCI expansion, which is $O(n^3)$. Therefore, the total online execution time is polynomial in n .

B.3.3 Part 3: Complexity of the Universal Decomposition Model. For a complex C-KPPQ decomposed into k sub-queries by the Universal Decomposition Model, the evaluation plan involves a sequence of $k - 1$ joins. Since each sub-query evaluation is polynomial (as shown above), and the number of joins k is determined by the query expression and is independent of the data size n , the overall execution remains polynomial. The cost model (Eq. 9-11) guides the join order but does not change the polynomial nature of the underlying operations. □

B.4 Discussion and Implications

This proof establishes that the computational bottleneck of KPPQ evaluation is shifted from the online phase (traditionally NP-complete) to the offline indexing phase (polynomial). The online phase is reduced to a series of lookups and expansions over a precomputed, polynomial-sized index.

The critical enabling factor is the *polynomial-sized NCI* and the *empirical completeness* of the representative paths. While the heuristic selection of paths does not guarantee the minimal possible index, it ensures that the index size does not explode exponentially, thus preserving the polynomial-time complexity of both construction and query execution.

This result fundamentally alters the complexity landscape for KPPQs, providing a "practically perfect" solution that bypasses theoretical intractability without sacrificing semantic precision.

B.5 Theoretical Implications and Comparison

The contrasting complexity results established in the previous subsections highlight the fundamental theoretical advancement of our approach:

Table 7: Complexity Comparison: Traditional vs. NCI-Based Approaches

Aspect	Traditional Methods	NCI-Based Approach
Time Complexity	$O(V^{nV+1})$ (Exponential)	$O(p^3 \cdot n^2 m)$ (Polynomial)
Space Complexity	Exponential	Polynomial
Online Execution	NP-complete	Polynomial-time
Semantics	Simple Path	Simple Path
Practical Feasibility	Limited	High

This theoretical comparison demonstrates that our NCI framework successfully bypasses the exponential complexity barrier while maintaining semantic fidelity—achieving what was previously considered theoretically intractable for general KPPQ evaluation under simple path semantics.

C NCI Maintenance Under Graph Updates

While the Node Clustered Index (NCI) is designed primarily for static graphs, real-world applications often require handling graph updates. This section outlines efficient maintenance strategies for NCI.

C.1 Update Impact Analysis

Graph updates affect NCI differently based on type:

- **Edge Updates:** May create/break connectivity patterns, affecting representative paths
- **Node Updates:** Impact Starting/End Node Sets and invalidate paths containing affected nodes
- **Batch Updates:** Cumulative effects may require comprehensive reconstruction

C.2 Maintenance Strategies

C.2.1 Incremental Maintenance. For small-scale updates, we update affected components locally:

- **Edge Addition:** Update SNS/ENS, extend tree paths, detect new cycles
- **Edge Removal:** Invalidate affected paths, update SNS/ENS accordingly

- **Node Operations:** Remove all paths containing deleted nodes

C.2.2 Lazy Validation. For query-time efficiency, we validate NCI components only when accessed:

- Maintain update log to track affected components
- Validate and update NCI segments during query processing
- Balance freshness and performance trade-offs

C.2.3 Periodic Reconstruction. When incremental updates become inefficient:

- Trigger full reconstruction when update ratio exceeds threshold
- Use background processes to minimize disruption
- Maintain multiple versions for seamless transition

C.3 Complexity and Practical Considerations

Table 8: Maintenance Operation Complexity

Operation	Time Complexity
Single Edge Update	$O(\lambda ^3 \cdot d_{max})$
Node Deletion	$O(\Sigma \cdot RP \cdot L_{avg})$
Incremental Batch	$O(\Delta E \cdot \lambda ^3 \cdot d_{max})$
Full Reconstruction	$O(p^3 \cdot n^2 m)$

Implementation Notes:

- Distributed maintenance by predicate domains for scalability
- Versioned NCI snapshots for temporal query support
- Memory management with LRU caching of frequent components

C.4 Limitations and Future Directions

Current approach is optimal for semi-static graphs. Challenges remain for:

- High-frequency update scenarios
- Cross-predicate dependency management
- Real-time streaming graph updates

Future research directions include extending the NCI framework to handle complex graph patterns such as nested cycles and overlapping cyclic structures, as well as optimizing index maintenance for highly dynamic graph environments.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009